

Formalización y resolución de problemas via SAT

Laboratorio 4 Lógica para Computación Otoño 2026

El objetivo de este laboratorio es poner en práctica una de las posibles aplicaciones de la lógica formal, específicamente en la resolución de problemas. A grandes rasgos, la idea es formalizar problemas enunciados en el lenguaje natural para hallar soluciones de forma algorítmica mediante herramientas de decisión de satisfacibilidad (SMT solvers).

1 Introducción

El objetivo de esta sección es servir como una breve introducción de conceptos básicos así como de algunas herramientas basadas en dichos conceptos.

1.1 Clasificación fundamental de problemas

En general nos interesa resolver problemas. Pero no cualquier tipo de problemas, sino aquellos que se puedan formular de manera matemáticamente precisa, diremos *problemas formales*. El interés en estos problemas no es solo práctico, además constituyen la base de la Teoría de la Computación.

Los problemas formales se pueden clasificar en tres categorías principales:

1. **Problemas de decisión.** Son aquellos cuya respuesta es “sí” o “no” (o sea, verdadero o falso). Ejemplos:
 - Determinar si una fórmula de LP es satisfacible (SAT).
 - Determinar si es posible colorear un grafo usando ciertos colores.
 - Determinar si existe un camino entre dos vértices de un grafo.
2. **Problemas de búsqueda.** Son aquellos que consisten en encontrar un objeto que cumpla ciertas propiedades especificadas. Dicho objeto puede no ser único. Ejemplos:
 - Encontrar una interpretación que satisface (un modelo) una fórmula de LP.
 - Encontrar un coloreo de un grafo usando ciertos colores.
 - Encontrar un camino entre dos vértices de un grafo.
3. **Problemas de optimización.** Son aquellos que consisten en encontrar el “mejor” objeto según algún criterio cuantificable. De nuevo, dicho objeto puede no ser único. Ejemplos:
 - Encontrar una interpretación que satisface la mayor cantidad de cláusulas de una fórmula en FNC.
 - Encontrar un coloreo de un grafo usando la mínima cantidad de colores (número cromático).
 - Encontrar el camino más corto entre dos vértices de un grafo.

Además, estas categorías se pueden relacionar de la siguiente manera dejando clara la existencia de una jerarquía entre ellas.

- Los problemas de optimización pueden reducirse a problemas de búsqueda. Por ejemplo, consideremos el problema de encontrar un coloreo de un grafo usando la mínima cantidad de colores. Supongamos un grafo (no vacío) con n vértices. Transformamos el problema de optimización a una serie de problemas de búsqueda sobre un intervalo de m colores comenzando con $m = 1$ y mientras m no sea suficiente incrementamos m sucesivamente hasta a lo sumo n :¹
 - Encontrar un coloreo del grafo usando $m = 1$ color.
 - Encontrar un coloreo del grafo usando $m = 2$ colores.
 - ⋮
 - Encontrar un coloreo del grafo usando $m = n$ colores.

Es claro que en general, para un problema de optimización dado, la cota inferior y superior para el intervalo de búsqueda depende del problema.

- Los problemas de búsqueda pueden reducirse a problemas de decisión. Por ejemplo, consideremos el problema de encontrar la posición de un elemento x en un array ordenado A . Podemos transformarlo en una serie de problemas de decisión de la forma:

¿Está el elemento x en la primera mitad del arreglo A ?

Si la respuesta es positiva, repetimos la pregunta para la primera mitad, de lo contrario repetimos la pregunta para la segunda mitad. Esto permite progresar dividiendo el espacio de búsqueda hasta encontrar el elemento o determinar que no existe. En esencia, este razonamiento explica como opera el muy conocido algoritmo de *búsqueda binaria*.

En vista de las reducciones anteriores, los problemas de decisión se pueden considerar fundamentales en la jerarquía. Sin embargo, por razones obvias, tendemos a enfocarnos en la búsqueda porque en la práctica queremos encontrar las soluciones de un problema y no solamente saber si ellas existen. En lo siguiente, cuando hablemos de *resolver* un problema vamos a referirnos a encontrar soluciones.

1.2 Lógica(s) y el problema de decisión de SAT

Además de las transformaciones entre categorías mencionadas antes, resulta que muchos problemas aparentemente distintos pueden transformarse entre sí. En este laboratorio vamos a resolver una variedad de problemas de búsqueda y optimización reduciéndolos siempre al problema de decisión y búsqueda de SAT. Podemos pensar en el problema de decisión de SAT como un problema de decisión universal.

Sea $\mathcal{L}$ un lenguaje formal de lógica y $\alpha \in \mathcal{L}$ una fórmula, el problema de decisión de SAT se puede formular como la pregunta

¿Existe una interpretación $\mathcal{I}$ de $\mathcal{L}$ que satisface α ?

o alternativamente en símbolos

$$\exists \mathcal{I} : \mathcal{I} \models \alpha$$

Algunas lógicas son más expresivas que otras, y es importante tener en cuenta que la computabilidad del problema depende de la misma, por ejemplo:

¹Asumiendo grafos planos, el [Teorema de los 4 colores](#) nos asegura que 4 colores son suficientes. Por lo tanto, bajo dicha hipótesis, podríamos dejar de buscar luego de 4.

- Si $\mathcal{L}$ es el lenguaje de la Lógica Proposicional (LP), sabemos que existe un algoritmo² que permite resolver la cuestión para cualquier $\alpha \in \mathcal{L}$ y por lo tanto decimos que la decisión de SAT en LP es decidible.
- Si $\mathcal{L}$ es un lenguaje de Lógica de Predicados (LPO), es demostrable que no existe un algoritmo que permite resolver la cuestión para cualquier $\alpha \in \mathcal{L}$ y por lo tanto decimos que la decisión de SAT en LPO es indecidible.³

1.3 SAT-solvers vs SMT-solvers, y el formato SMT-LIB

Distinguimos aquí entre SAT-solvers y SMT-solvers. Los primeros sirven solo para lógica pura (específicamente LP), y los segundos extienden las capacidades de los primeros admitiendo el uso de cuantificadores y otras facilidades.

SAT-solver. Por SAT-solver nos referimos a una herramienta computacional que resuelve problemas de SAT exclusivamente en LP. Este tipo de herramienta es al fin y al cabo una implementación muy optimizada de algún algoritmo de resolución de SAT (como lo podría ser por ejemplo el Método de Tableaux) siendo capaz de manejar cientos o incluso miles de variables⁴. Para este tipo de herramientas, la respuesta solo puede ser **sat** o **unsat**. En el caso **sat** es posible además pedirle que exhiba un modelo.

SMT-solver. Un SMT-solver (*Satisfiability modulo theories*) es una herramienta computacional que generaliza el problema de resolver SAT de LP a teorías de LPO que describen una variedad de objetos como números naturales, números enteros, conjuntos, arrays, strings, etc. En particular, un SMT-solver también es un SAT-solver, por lo que internamente depende de algún procedimiento de decisión de SAT. Estas herramientas pueden ser usadas para verificar mecánicamente proposiciones matemáticas. En general, el razonamiento sobre fórmulas arbitrarias es indecidible y por este motivo la respuesta de un SMT-solver puede ser **sat**, **unsat** o **unknown**.

Algunos SMT-solvers muy conocidos son Z3, CVC5 y Yices. En este laboratorio proponemos el uso de CVC5 simplemente por ser *open-source*. Para nuestros propósitos, será usado solo como SAT-solver, o sea que haremos un uso relativamente liviano de sus capacidades. Tiene disponible una [interface online](#) que puede ser ocasionalmente útil pero vamos a preferir usarlo de forma local (ver [guía de instalación](#)).

SMT-LIB. Antiguamente, cada SMT-solver tenía su propio formato de entrada y salida, dificultando la colaboración entre usuarios de distintas herramientas. Esto motivó la creación de SMT-LIB, un estándar para el formato de entrada y salida de SMT-solvers, entre otras cosas. Las tres herramientas mencionadas previamente (Z3, CVC5, Yices) soportan este estándar, por lo que son en gran parte compatibles entre sí.

SMT-LIB define un lenguaje de lógica usando una sintaxis estilo LISP con *S-expressions* por lo cual todos los operadores aparecen prefijos y las expresiones con espacios quedan siempre

²Por ejemplo el Método por Tabla de verdad o el Método de Tableaux como fue visto en la primera parte del curso. Además, notar que la semántica de LP es un álgebra de Boole, por lo cual resolver SAT es en definitiva hallar una asignación de valores sobre variables booleanas para que una expresión sea igual a 1 (p.ej. resolver SAT de $\neg p \vee q$ es lo mismo que en álgebra resolver $\bar{p} + q = 1$).

³Este resultado negativo no significa que LPO es inutilizable. Podemos en LPO usar el Método de Tableaux sobre una gran cantidad de fórmulas de interés sin problema.

⁴Considerar que para este problema no se conoce hasta la fecha una manera mejor de resolverlo que no tenga complejidad computacional exponencial en el peor caso (cuestión estrechamente vinculada al problema abierto **P vs NP**). Las mejores implementaciones que tenemos disponibles son el resultado de décadas de esfuerzo.

parentizadas. Por ejemplo, la fórmula $p \supset q \wedge r$ de $\mathcal{L}_{LP}$ se escribe $(\Rightarrow p \text{ (and } q \text{ } r))$ en SMT-LIB. En general, no se trata de una sintaxis muy conveniente para ser usada directamente a mano porque fue pensada para ser generada de forma programática a partir de interfaces disponibles para cada lenguaje de programación. Presentaremos en la siguiente sección un módulo de Haskell que servirá para tal fin.

Si bien en la práctica el usuario no escribe en el formato de SMT-LIB directamente, es instructivo saber (aunque solo sea por razones de *debugging*) que el formato de entrada para el SAT-*solver* consiste en un script que en general puede tener la forma de la figura 1.⁵

```

0 (set-logic LOGIC)           ; elegir una lógica, p.ej. QF_UF
1 (set-option :produce-models true) ; opción para generar modelo
2
3 ; Declaración de símbolos
4
5 (assert  $\alpha_1$ )
6 (assert  $\alpha_2$ )
7 ...
8 (assert  $\alpha_n$ )
9
10 (check-sat)                ; decidir SAT
11 (get-model)                 ; obtener modelo en caso positivo

```

Figura 1: Ejemplo de script genérico de SMT-LIB.

Este script ordena al SAT-*solver* usar una lógica determinada, declara o define los símbolos del vocabulario a usar y luego afirma una o más fórmulas α_i para resolver la satisfacibilidad de

$$\alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_n$$

1.4 Presentación del módulo `SMTlib.hs`

El módulo `SMTlib.hs` incluido en este laboratorio tiene como objetivo facilitar la comunicación con cualquier SMT-*solver* compatible con SMT-LIB. El uso de este módulo (junto al tipo `L` y funciones auxiliares) servirán como interfase entre el programador y el SMT-*solver* facilitando la creación de scripts y la interacción con dicha herramienta.

Flujo de ejecución. El flujo de ejecución al trabajar con este módulo es ilustrado en la figura 2. El módulo provee dos funciones principales que sirven para invocar al SMT-*solver*:

```

decide :: SATproblem → IO Bool
solve  :: SATproblem → IO (Maybe Model)

```

La única diferencia es que mientras la primera solo decide SAT (responde `Bool`), la segunda además posiblemente devuelve un modelo (responde `Maybe Model`). Si el resultado es SAT, obtenemos `Just m` (donde `m` es un modelo) y en caso contrario obtenemos `Nothing`.

El tipo `Model` se define como `[(String,String)]` representando parejas (símbolo, valor). Al trabajar en LP el valor será un booleano `true` o `false`.

Nos concentraremos aquí en la función `solve` ya que será nuestra manera de resolver el problema de búsqueda de SAT.

¿Qué hace `solve`? Será suficiente saber que la misma debe “salir” del ambiente de ejecución de Haskell para solicitarle al sistema subyacente:

⁵Un tutorial de SMT-LIBv2 se puede ver [aquí](#), sin embargo no lo consideramos una lectura necesaria.

1. La creación de un archivo `script.smt2` en el directorio de ejecución actual (nombre y ubicación por defecto) donde se guarda el script SMT-LIB (ver figura 1) con el problema a resolver.
2. La creación de un proceso para ejecutar el SMT solver pasándole la ubicación del archivo `script.smt`. Del resultado de esta ejecución obtenemos la respuesta `Maybe Model`.

¿Por qué el retorno es de tipo `IO`? Como Haskell es un lenguaje de programación funcional *puro* (no hay estado ni efectos), la manera de interactuar con el mundo exterior es a través de *mónadas*. La consecuencia es que el resultado de la invocación viene dentro de la mónada `IO` (*Input Output*), y por eso el tipo de salida es `IO (Maybe Model)` y no simplemente `Maybe Model`. En otras palabras, una función como `solve` que pretende salir al mundo exterior (donde hay estado y efectos) se considera impura y por esto su tipo de salida quedará forzosamente encapsulado por `IO`. No vamos a entrar en más en detalle sobre las mónadas en este laboratorio ya que no será necesario y además es una característica más bien específica de lenguajes de programación como Haskell.

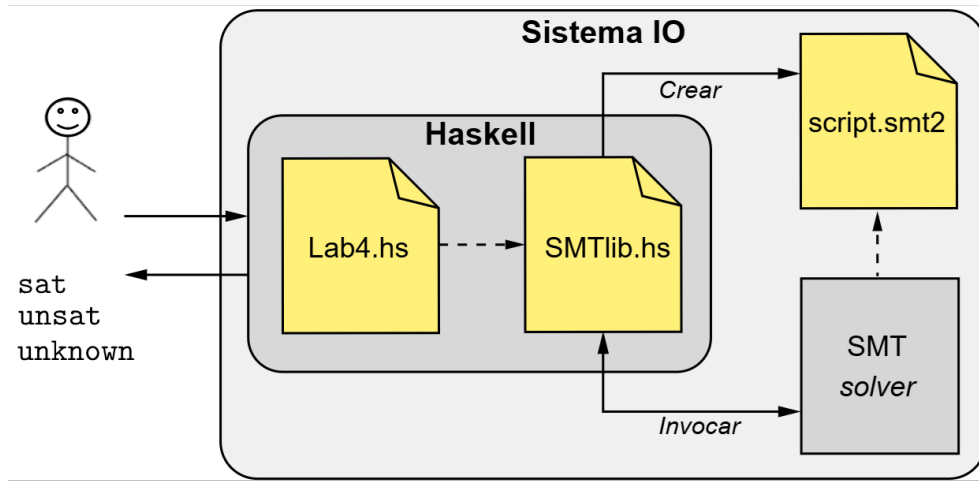


Figura 2: Flujo de ejecución. Las flechas punteadas representan dependencias. Las Flechas solidas representan acciones. La posición en la que se encuentra el módulo `SMTlib.hs` en el diagrama sugiere que se trata de una interfase entre el trabajo de nuestro laboratorio y la herramienta a utilizar. Por “Sistema IO” entendemos aquí cualquier ambiente de ejecución (interpretado o nativo) dentro del que se está ejecutando nuestro código Haskell.

Tipo `SATproblem`. El único parámetro de la función `solve` es de tipo `SATproblem`, esto es una especificación completa del problema de SAT a resolver. Se define

$$\text{SATproblem} = (\text{Logic}, [\text{SymDecl}], [\text{Formula}])$$

donde

- **Logic.** Es la lógica a usar, según estándar SMT-LIB. En este laboratorio vamos a usar siempre `QF_UF`, que sirve para hacer lógica proposicional.
- **[SymDecl].** Una lista de declaraciones de símbolos del vocabulario. En este laboratorio, aquí ponemos las letras/variables de lógica proposicional usadas para formalizar problemas.
- **[Formula].** Una lista de fórmulas cuya conjunción se pretende analizar.

Todos estos componentes mencionados son por simplicidad codificados como *strings* y deben respetar el estándar SMT-LIB. Tenemos disponibles funciones auxiliares que nos facilitan la especificación de estos componentes. Algunas son:

- **lp2SMT :: L → Formula**
Traduce una fórmula de $\mathcal{L}_{LP}$ (tipo L) a la sintaxis de SMT-LIB.
- **varDecl :: Type → String → SymDecl**
Genera la declaración de una variable con un tipo y un nombre.
- **genVars1 :: Type → String → [Int] → [SymDecl]**
Genera una lista con todas las posibles declaraciones de variables con tipo y nombre indexadas en un intervalo.

Ejemplo. Finalizamos esta introducción mostrando un ejemplo simple de como usar el módulo para determinar la satisfacibilidad de una fórmula:

$$(p_1 \supset p_2) \wedge (p_2 \supset p_3) \wedge (p_1 \vee \neg p_3) \quad (1)$$

Aquí identificamos tres letras proposicionales: p_1 , p_2 y p_3 . La figura 3 muestra la formalización en Haskell del vocabulario y la fórmula. Nuestra intención es interpretar todas las letras como variables booleanas.

```

1  -- Vocabulario: declaracion de tres variables booleanas
2  voc :: [SymDecl]
3  voc = genVars1 "Bool" "p" [1..3]
4
5  -- Formula como tipo L
6  form :: L
7  form = (p1 ==> p2) /\ (p2 ==> p3) /\ (p1 \/ neg p3)
8  where
9      p1 = v1 "p" 1
10     p2 = v1 "p" 2
11     p3 = v1 "p" 3

```

Figura 3: Código Haskell formalizando el problema de resolver la satisfacibilidad de 1.

Para resolver 1, invocamos al SMT-*solver* indicando la lógica, el vocabulario y la fórmula correspondiente (traducida con lp2SMT):

```

ghci> solve ("QF_UF", voc, [lp2SMT form])
Just [("p1","false"), ("p2","false"), ("p3","false")]

```

Así obtuvimos una de las soluciones posibles: $p_1 = p_2 = p_3 = false$. Esta invocación por detrás ha generado el script de la figura 4. Este se puede subir a la [interface online](#) para ver el resultado.

```

0  (set-logic QF_UF)
1  (set-option :produce-models true)
2
3  (declare-fun p1 () Bool)      ; Declaración de variables
4  (declare-fun p2 () Bool)
5  (declare-fun p3 () Bool)
6
7  (assert (and (=> p1 p2) (and (=> p2 p3) (or p1 (not p3)))))
8
9  (check-sat)
10 (get-model)

```

Figura 4: Script de SMT-LIB generado por el módulo SMTlib.hs a partir del código de la figura 3.

2 Ejercicios

1. (**Veraces y mentirosos**). En una extraña isla, viven tan sólo dos tipos de habitantes: los *caballeros*, que siempre dicen la verdad, y los *escuderos*, que siempre mienten. Además, los forasteros no pueden distinguir el tipo de los habitantes a simple vista.

En una ocasión, usted visita la isla y se encuentra con tres habitantes A , B y C , entablando un diálogo. Hay distintas versiones de como fue exactamente dicho diálogo, las cuales se presentan a continuación. En cualquier caso, el objetivo es determinar el tipo de los tres habitantes.

- 1.
- A : Todos somos escuderos.
 B : Solo uno de nosotros es caballero.
- 2.
- A : B y C son de distinto tipo.
 B : A no es caballero y yo soy escudero.
 B : C es escudero.
 C : A es caballero.

Se pide lo siguiente:

- Declarar el vocabulario de símbolos que se usará en la formalización, incluyendo un comentario de cómo se interpretan. Esto quedará expresado mediante la función:

vocEj1 :: [SymDecl]

- Completar la formalización de cada caso mediante las funciones:

caso1 :: [L]

caso2 :: [L]

- Para cada caso completar el comentario con la respuesta al problema, según el resultado que se obtiene de invocar al *SMT-solver*.

Sugerencia. En general, pensar:

- ¿Bajo qué condiciones la declaración de un habitante es verdadera?.
- ¿Bajo qué condiciones la declaración de un habitante es falsa?.
- ¿Cómo podemos expresar dichas condiciones formalmente?

2. **(Cardinalidad).** Los cuantificadores cardinales son cuantificadores que no solo dicen que existe algo, sino cuántos elementos cumplen una propiedad. Por ejemplo:

- $(\exists^{\geq k} x)\alpha(x)$ significa **Al menos** k individuos cumplen la propiedad α .
- $(\exists^{\leq k} x)\alpha(x)$ significa **A lo sumo** k individuos cumplen la propiedad α .
- $(\exists^k x)\alpha(x)$ significa **Exactamente** k individuos cumplen la propiedad α .

En lógica de predicados estándar no suelen tomarse como primitivos, pero se pueden definir usando cuantificadores ordinarios, igualdad y desigualdad.

En este laboratorio, nos concentraremos en dominios finitos representados por un conjunto de índices $I = \{1, 2, \dots, n\}$ (escrito en Haskell como $[1..n]$). Aquí proponemos dos formalizaciones distintas de estos cuantificadores en lógica proposicional.

1. Formalización ingenua.

a) Definir $(\exists^{\geq k} x)\alpha(x)$ según la meta-fórmula

$$(\exists i_1 \in I)(\exists i_2 \in I) \dots (\exists i_k \in I) \left(\bigwedge_{1 \leq a < b \leq k} i_a \neq i_b \wedge \bigwedge_{a=1}^k \alpha(i_a) \right)$$

que expresa “Hay k índices distintos y todos ellos cumplen α ”. La solución quedará implementada mediante la función de Haskell:

existAtLeast :: Nat → [Nat] → (Nat → L) → L

b) Definir $(\exists^{\leq k})$ en términos de $(\exists^{\geq k})$.

existAtMost :: Nat → [Nat] → (Nat → L) → L

c) Definir $(\exists^k)$ usando los puntos anteriores.

existExact :: Nat → [Nat] → (Nat → L) → L

2. Formalización en la forma de Sinz.

a) Definir $(\exists^{\leq k})$ usando la codificación de Sinz.

existAtMost' :: Nat → [Nat] → (Nat → L) → L

b) Definir $(\exists^{\geq k})$ en términos de $(\exists^{\leq k})$.

existAtLeast' :: Nat → [Nat] → (Nat → L) → L

c) Definir $(\exists^k)$ usando los puntos anteriores.

existExact' :: Nat → [Nat] → (Nat → L) → L

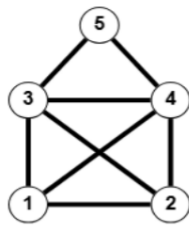
3. El tamaño de la formalización tiene un gran impacto en la eficiencia de la decisión de SAT. El tamaño de las fórmulas producidas por ambas propuestas depende fundamentalmente de los parámetros n y k .

- Analizar el tamaño de $(\exists^k)$ para la versión ingenua.
- Analizar el tamaño de $(\exists^k)$ para la versión Sinz.
- Comparar como crece el tamaño de ambas cuando fijamos n y variamos k .
- De acuerdo a sus resultados, comente cuando preferiría usar una versión sobre la otra.

3. **(Clique de grafo).** Un *grafo* (no dirigido) $G = (V, A)$ es una estructura formada por un conjunto finito de vértices V y un conjunto finito de aristas E entre vértices. Cuando existe una arista entre dos vértices decimos que son adyacentes. En nuestra codificación de grafos, siempre vamos a identificar los vértices V enumerándolos en el rango $1..|V|$ y representamos las aristas E mediante una lista de tipo `[(Nat, Nat)]`, entendida como una relación binaria dada por extensión, donde cada pareja (v_1, v_2) nos dice que v_1 y v_2 son adyacentes. Como el grafo es no dirigido, siempre que tenemos (v_1, v_2) también tenemos (v_2, v_1) . Por lo tanto, un grafo quedará representado en Haskell como un objeto de tipo

```
type Graph = (Nat, [(Nat, Nat)])
```

Por ejemplo, el grafo de la figura 5 tiene 5 vértices y 8 aristas, y se usan $8 \times 2 = 16$ parejas para codificar las aristas.



```
(5, [(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4), (3, 5), (4, 5),  
      (2, 1), (3, 1), (4, 1), (3, 2), (4, 2), (4, 3), (5, 3), (5, 4)])
```

Figura 5: A la izquierda un grafo en su representación habitual como diagrama y a la derecha nuestra codificación del mismo como un objeto de tipo `(Nat, [(Nat, Nat)])`. El orden en el que se listan las parejas es irrelevante y se asume que no se repiten.

Un *clique* de un grafo es un subconjunto de vértices del grafo tal que todo par de vértices distintos son adyacentes. En otras palabras, un clique es un subgrafo del grafo original en el que cada vértice está conectado a todos los demás vértices del subgrafo (un clique siempre es un grafo completo).

En ciencias de la computación, el *problema del k -clique* consiste en dado un grafo, decidir si existe en este un clique formado por k vértices (decimos tamaño k). Cuando identificamos un clique es conveniente describirlo listando los vertices que lo conforman. Por ejemplo, en el grafo de la figura 5 tenemos:

- Cinco 1-cliques: $[1]$, $[2]$, $[3]$, $[4]$ y $[5]$.
- Ocho 2-cliques: $[1, 2]$, $[1, 3]$, $[1, 4]$, $[2, 3]$, $[2, 4]$, $[3, 4]$, $[3, 5]$ y $[4, 5]$.
- Cuatro 3-cliques: $[1, 2, 3]$, $[1, 2, 4]$, $[2, 3, 4]$ y $[3, 4, 5]$.
- Un 4-clique: $[1, 2, 3, 4]$. Este es el clique más grande.
- Ningún 5-clique.

En general, una instancia de problema del k -clique es representada en Haskell usando el tipo

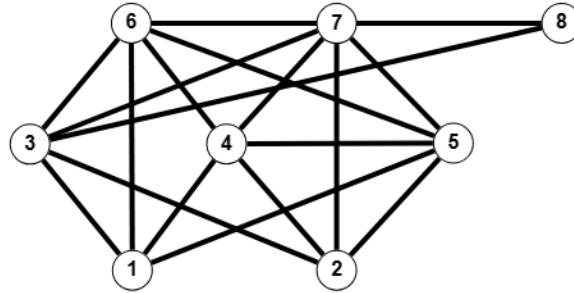
```
type KClique = (G, [Nat])
```

Ejercicios:

1. Formalizar el problema de búsqueda del k -clique. La respuesta quedará expresada mediante las funciones:

$$\begin{aligned} \text{kClique} &:: \text{KClique} \rightarrow [\text{L}] \\ \text{solveKClique} &:: \text{KClique} \rightarrow \text{IO (Maybe Model)} \end{aligned}$$

2. Considere el siguiente grafo



Usando la parte anterior, responder las siguientes preguntas:

- a) ¿Existe un 3-clique?. En caso afirmativo, exhibirlo.
 - b) ¿Existe un 5-clique?. En caso afirmativo, exhibirlo.
 - c) ¿Cuál es el clique más grande?.
3. El problema correspondiente de optimización consiste en maximizar k y por lo tanto encontrar el clique más grande posible. Evidentemente, este tendrá a lo sumo el mismo tamaño que el grafo dado.
 - a) Implementar en términos de **solveKClique**:

$$\text{maxClique} :: \text{KClique} \rightarrow \text{IO (Maybe (Int, Model))}$$

- b) Confirmar el clique más grande encontrado en la parte 2.c.
 - c) Considere el siguiente resultado de teoría de grafos:
Teorema. Sea $G = (V, E)$ un grafo no dirigido, y sea $\Delta(G)$ el grado máximo de G . Si G contiene un k -clique, entonces $k \leq \Delta(G) + 1$.
 Convénzase de que es válido. ¿Cómo podría explotar este resultado en la implementación anterior?
4. El problema del k -clique tiene aplicaciones en bioinformática, química computacional y otras ciencias.⁶ En aplicaciones reales es común trabajar con grafos muy grandes almacenados en disco o bases de datos. Aquí vamos a adaptar la implementación de ejercicios anteriores para leer grafos desde archivos en formato TSV⁷.
 - a) Implementar la nueva versión de **solvekClique**:

$$\text{solvekCliqueFrom} :: \text{String} \rightarrow \text{Nat} \rightarrow \text{IO (Maybe Model)}$$

⁶Por ejemplo identificación y predicción de proteínas asociadas a enfermedades, estudio de la organización del genoma, representación de estructuras moleculares y sus interacciones para el desarrollo de fármacos, análisis de interacciones en redes sociales, entre otros.

⁷TSV contiene datos organizados en filas y columnas, donde cada fila representa un registro y cada columna un campo de datos. Los valores dentro de cada fila se separan mediante tabulaciones.

b) Implementar la nueva versión de `maxClique`:

```
maxCliqueFrom :: String → IO (Maybe Model)
```

5. Determinar el clique más grande en los siguientes grafos.⁸

- a) **network_yeast.tsv** es una versión del grafo de la levadura (ilustrado en 6). Este grafo modela las interacciones entre proteínas en la levadura común (*Saccharomyces cerevisiae*). Los grupos densamente interconectados de proteínas se pueden identificar mediante cliques en el grafo y sugieren funciones específicas dentro de la célula que son de interés para los especialistas en biología.
- b) **network_butterfly.tsv** es un grafo donde los nodos son mariposas y las aristas representan similitud visual entre mariposas, calculada a partir de imágenes de mariposas. Los cliques son útiles para detectar grupos homogéneos de mariposas e identificar patrones comunes de mimetismo.

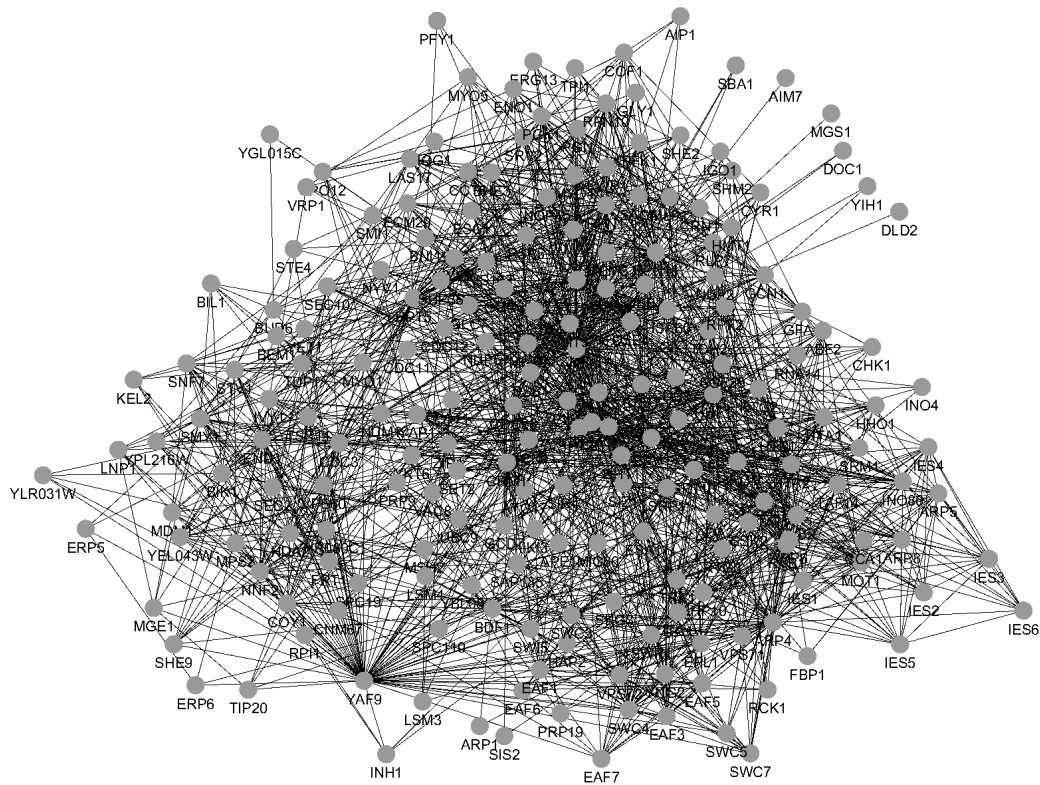


Figura 6: Interacción de proteínas en *Saccharomyces cerevisiae* (levadura). En su forma original, este grafo tiene 209 vértices (proteínas) y 1776 aristas (interacciones). Sirve para modelar cómo las proteínas se conectan y comunican dentro de la célula.

⁸Los datos son tomados del [Stanford Large Network Dataset Collection](#).

4. **(Camino Hamiltoniano).** Un camino hamiltoniano en un grafo es un recorrido que pasa por todos los vértices exactamente una vez. Es decir, si el grafo representa ciudades, personas, especies o estados, un camino hamiltoniano sería una forma de visitarlos todos sin repetir ninguno, siguiendo solamente las aristas existentes y pudiendo pasar por la misma arista más de una vez.

Ejercicios:

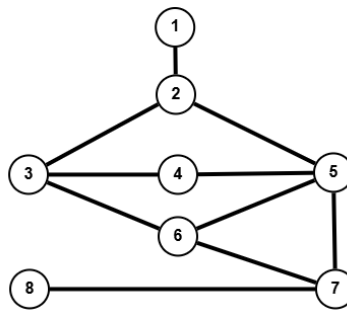
1. Formalizar el problema del camino Hamiltoniano. La respuesta quedará expresada mediante las funciones:

```

hamilPath :: Graph → [L]
solveHamilPath :: Graph → IO (Maybe Model)

```

2. Considere el siguiente grafo



Usando la parte anterior, encontrar dos caminos hamiltonianos (no inversos).

3. En general, decimos que un camino $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_k$ es un *ciclo* si $v_1 = v_k$ y, salvo esa repetición del inicio/final, no se repiten vértices.

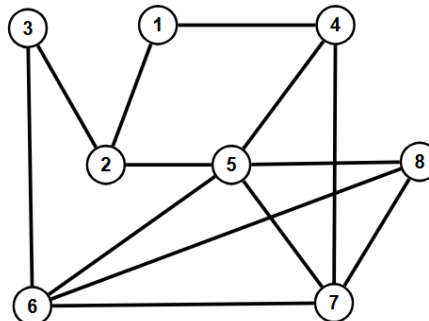
Se pide formalizar el problema del ciclo Hamiltoniano. La respuesta quedará expresada mediante las funciones:

```

hamilCycle :: Graph → [L]
solveHamilCycle :: Graph → IO (Maybe Model)

```

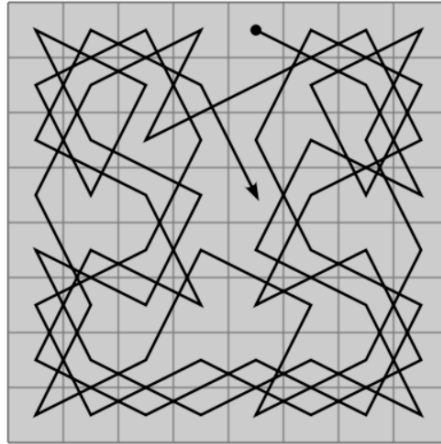
4. Considere el siguiente grafo



Usando la parte 1, responder las siguientes preguntas:

- a) ¿Tiene este grafo un ciclo hamiltoniano?. En caso afirmativo, exhibirlo.
- b) ¿Tiene el grafo del ejercicio 2 un ciclo hamiltoniano?. En caso afirmativo, exhibirlo.

5. El *problema del caballero* (o *el caballo*), en su forma abierta, consiste en encontrar un recorrido de un caballo de ajedrez sobre un tablero de $n \times n$, de modo que visite cada casilla exactamente una vez. La siguiente figura ilustra un posible camino en un tablero de tamaño clásico.



Si además el caballo puede volver desde la última casilla a la primera, se obtiene lo que denominamos un *recorrido cerrado*.

- a) Resolver el problema del caballero en su forma abierta. Implementar:

`solveKnightTour :: Nat → IO (Maybe Model)`

- b) Resolver el problema del caballero en su forma cerrada. Implementar:

`solveClosedKnightTour :: Nat → IO (Maybe Model)`

- c) Usando las partes anteriores, intentar completar con Sí/No:

n	Hay recorrido abierto?	Hay recorrido cerrado?
1		
2		
3		
4		
5		
6		
7		
8		

- d) Igual que b), pero a partir de una casilla inicial dada. Implementar:

`solveClosedKnightTourFrom :: (Nat,Nat) → Nat → IO (Maybe Model)`

- e) ¿Hay recorrido cerrado comenzando en la casilla $(1,1)$ para $n = 8$?

Sugerencia. Se recomienda implementar las siguientes funciones de soporte que le permitirán construir en Haskell operadores sobre intervalos.

- **bigAnd** :: [Nat] → (Nat → L) → L

Dada una lista de índices I y una fórmula indexada α , devuelve la conjuntaria de α sobre I , o sea $\bigwedge_{i \in I} \alpha_i$.

Ejemplo: Para representar $\bigwedge_{i \in [1,2,3]} p_i$ en Haskell escribimos

`bigAnd [1, 2, 3] (\lambda i \rightarrow v ("p"++show i)) = (v "p1") And (v "p2") And (v "p3")`

- **bigOr** :: [Nat] → (Nat → L) → L

Dada una lista de índices I y una fórmula indexada α , devuelve la disyuntoria de α sobre I , o sea $\bigvee_{i \in I} \alpha_i$.

Ejemplo: Para representar $\bigvee_{i \in [1,2,3]} p_i$ en Haskell escribimos

`bigOr [1, 2, 3] (\lambda i \rightarrow v ("p"++show i)) = (v "p1") Or (v "p2") Or (v "p3")`

Ilustramos con un ejemplo como se pueden combinar algunas de las funciones anteriores en el contexto de LP. La conjunción de disyunciones de literales indexados (una FNC) sobre $I = [1..n]$ y $J = [1..m]$:

$$\bigwedge_{i \in I} \bigvee_{j \in J} p_{i,j}$$

es representada en Haskell escribiendo

`bigAnd [1..n] (\lambda i \rightarrow bigOr [1..m] (\lambda j \rightarrow v2 "p" i j))`

En particular, si $n = 2$ y $m = 2$, esto construye en Haskell la fórmula de LP

`Bin (Bin (v "p1_1") Or (v "p1_2")) And (Bin (v "p2_1") Or (v "p2_2"))`

que en la sintaxis de SMT-LIB se escribe

`(and (or p1_1 p1_2) (or p2_1 p2_2))`

y en nuestra sintaxis concreta (“notación de mano alzada”) se escribe

$(p_{1,1} \vee p_{1,2}) \wedge (p_{2,1} \vee p_{2,2})$